



AI Security

- Lecture 04. Dimension Reduction Techniques

Artificial Intelligence Research (AIR) LAB

<https://air.korea.ac.kr/>

School of Cybersecurity

Korea University

Sangkyun Lee (이상근)

2025. Spring Semester

차원 축소의 필요성

많은 머신러닝 문제는 각 훈련 인스턴스에 대해 수천 또는 수백만 개의 특성을 포함함. 이러한 모든 특성은 훈련을 매우 느리게 할 뿐만 아니라 좋은 솔루션을 찾기 어렵게 만듦. 이 문제를 종종 '차원의 저주'라고 함.

다행히도 실제 문제에서는 특성의 수를 상당히 줄이는 것이 가능하여 다루기 어려운 문제를 다룰 수 있는 문제로 바꿀 수 있음. 예를 들어, MNIST 이미지의 경우 이미지 테두리의 픽셀은 거의 항상 흰색이므로 많은 정보 손실 없이 훈련 세트에서 이러한 픽셀을 완전히 제거할 수 있음.

차원 축소는 JPEG로 이미지를 압축하는 것처럼 일부 정보 손실을 유발함. 훈련 속도를 높이지만 시스템 성능이 약간 저하될 수 있음. 파이프라인도 더 복잡해짐. 따라서 차원 축소를 고려하기 전에 먼저 원본 데이터로 시스템을 훈련해 볼 것을 권장함.

차원 축소의 주요 접근법

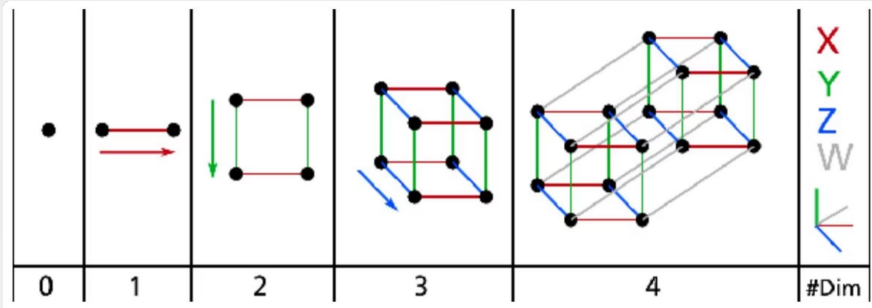
이 장에서는 먼저 차원의 저주에 대해 논의하고 고차원 공간에서 어떤 일이 일어나는지 이해함. 그런 다음 차원 축소의 두 가지 주요 접근 방식(투영 및 매니폴드 학습)을 살펴보고, 세 가지 인기 있는 차원 축소 기법을 살펴볼 것임:

 **PCA (주성분 분석)**
데이터의 분산을 최대한 보존하는 방향으로 차원을 축소하는 기법

 **랜덤 투영**
무작위 선형 투영을 사용하여 데이터를 저차원 공간으로 투영하는 방법

 **LLE (지역 선형 임베딩)**
비선형 차원 축소 기법으로, 데이터의 지역적 관계를 보존하며 차원을 축소함

차원의 저주



우리는 3차원 공간에 살면서 직관을 형성했기 때문에 고차원 공간을 상상하려고 할 때 직관이 실패함. 기본적인 4D 초입방체조차 우리 마음속에 상상하기 어려움.

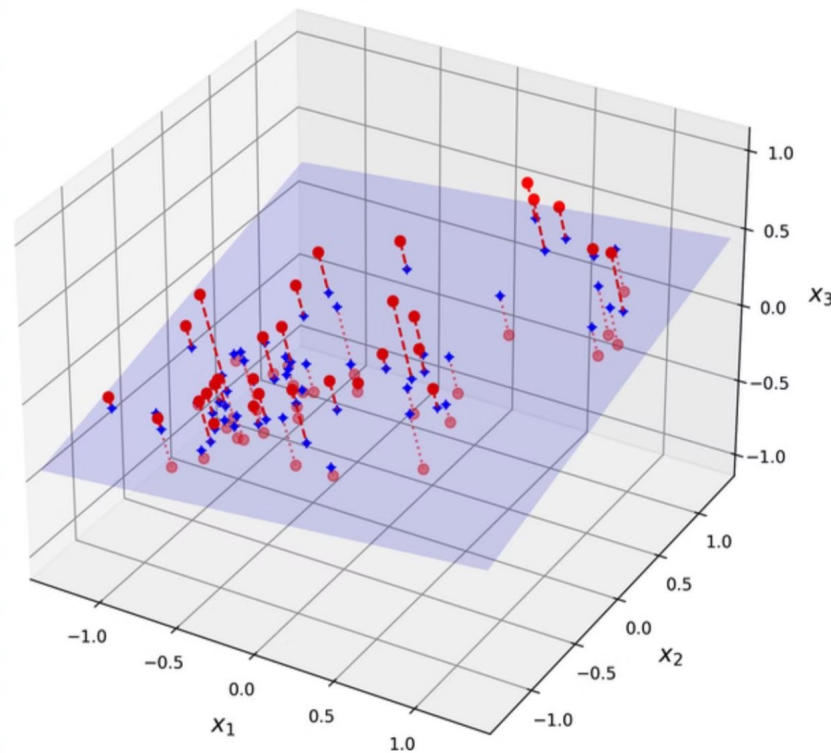
고차원 공간에서는 많은 것들이 매우 다르게 작동함. 예를 들어, 단위 정사각형(1×1 정사각형)에서 무작위로 점을 선택하면 경계에서 0.001 미만의 거리에 위치할 확률이 약 0.4%에 불과함. 그러나 10,000차원 단위 초입방체에서는 이 확률이 99.999999%보다 큼. 고차원 초입방체에서 대부분의 점은 경계에 매우 가까움.

더 문제가 되는 차이: 단위 정사각형에서 무작위로 두 점을 선택하면 두 점 사이의 거리는 평균적으로 약 0.52임. 3D 단위 큐브에서는 평균 거리가 약 0.66임. 그러나 1,000,000차원 단위 초입방체에서 무작위로 선택한 두 점의 평균 거리는 약 408.25임!

차원 축소의 주요 접근법: 투영

대부분의 실제 문제에서 훈련 인스턴스는 모든 차원에 균일하게 분포되어 있지 않음. 많은 특성이 거의 일정하거나 다른 특성과 높은 상관 관계가 있음. 결과적으로 모든 훈련 인스턴스는 고차원 공간의 훨씬 낮은 차원 부분공간 내(또는 가까이)에 위치함.

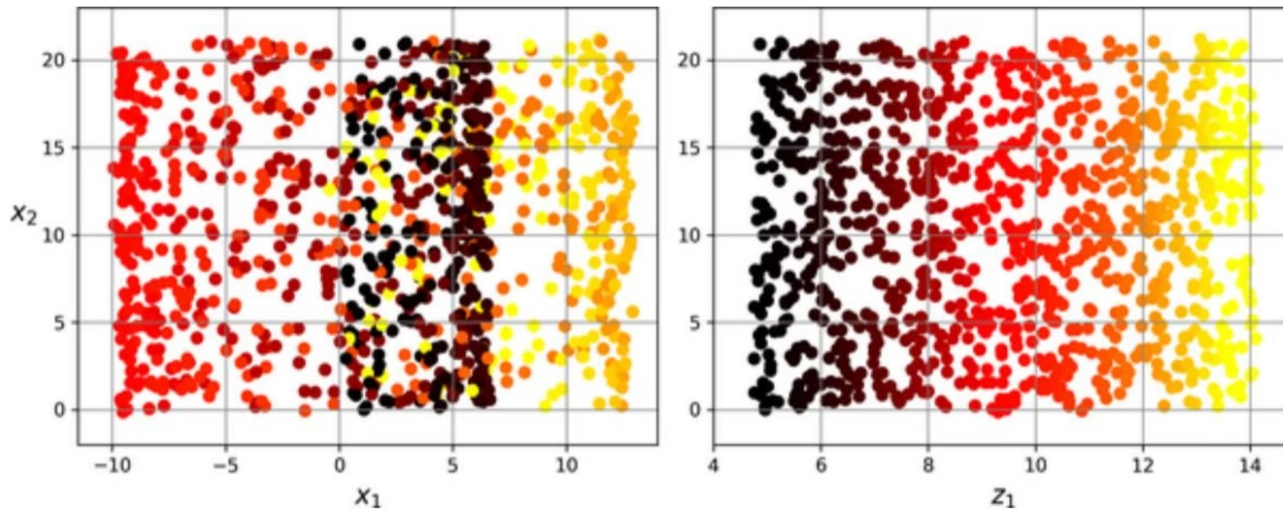
모든 훈련 인스턴스가 평면에 가깝게 위치함: 이는 고차원(3D) 공간의 저차원(2D) 부분공간임. 모든 훈련 인스턴스를 이 부분공간에 수직으로 투영하면 새로운 2D 데이터셋이 생성됨.



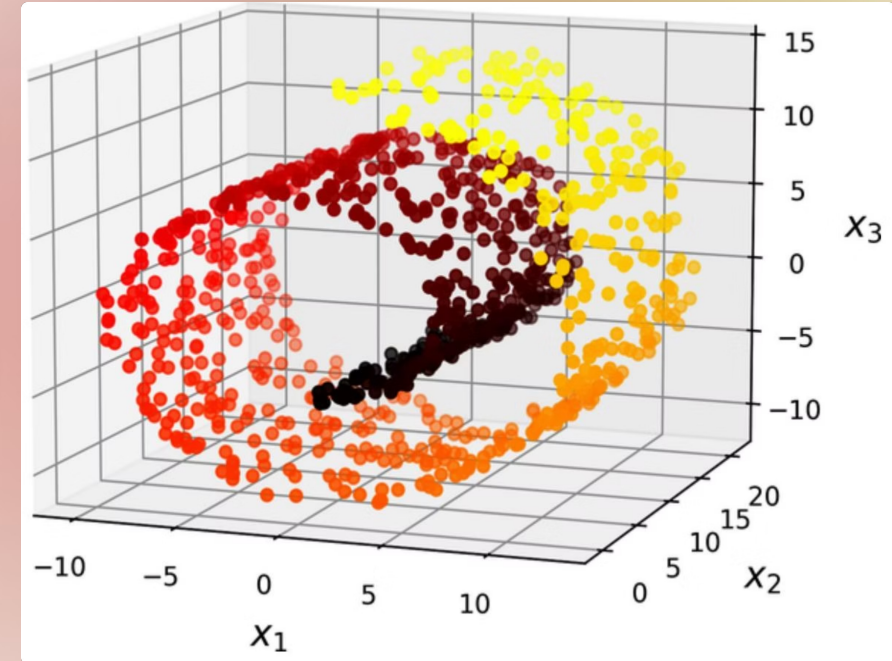
매니폴드 학습

투영이 항상 차원 축소에 가장 좋은 접근법은 아님. 많은 경우 부분공간이 비틀리고 구부러질 수 있음. 스위스 롤 데이터셋이 그 예임.

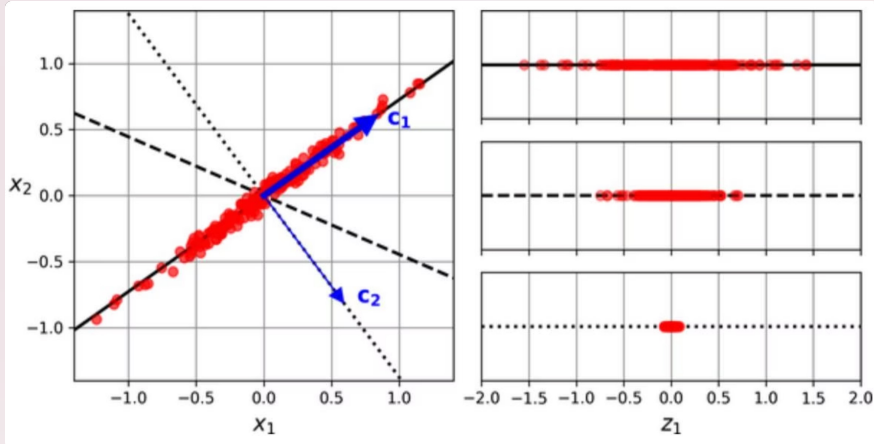
단순히 평면에 투영하면 스위스 롤의 다른 층이 함께 압축됨. 대신 스위스 롤을 펼쳐서 2D 데이터셋을 얻는 것이 바람직함.



스위스 롤은 2D 매니폴드의 예임. 간단히 말해, 2D 매니폴드는 고차원 공간에서 구부러지고 비틀릴 수 있는 2D 형태임. 더 일반적으로, d 차원 매니폴드는 n 차원 공간의 일부로, 국소적으로 d 차원 초평면과 유사함.



PCA (주성분 분석)



주성분 분석(PCA)은 가장 인기 있는 차원 축소 알고리즘임. 먼저 데이터에 가장 가까운 초평면을 식별한 다음 데이터를 그 위에 투영함.

분산 보존

훈련 세트를 저차원 초평면에 투영하기 전에 먼저 올바른 초평면을 선택해야 함. 그림에서 볼 수 있듯이, 실선에 대한 투영은 최대 분산을 보존하는 반면, 점선에 대한 투영은 매우 적은 분산을 보존함.

최대 분산을 보존하는 축을 선택하는 것이 합리적임. 이는 원본 데이터셋과 그 축에 대한 투영 사이의 평균 제곱 거리를 최소화하는 축이기도 함.

주성분

PCA는 훈련 세트에서 가장 큰 분산을 차지하는 축을 식별함. 또한 첫 번째 축에 직교하고 남은 분산 중 가장 큰 부분을 차지하는 두 번째 축을 찾음. 고차원 데이터셋이라면 PCA는 세 번째, 네 번째 등 데이터셋의 차원 수만큼 축을 찾음.

i번째 축은 데이터의 i번째 주성분(PC)이라고 함. 주성분을 찾는 방법은 특이값 분해(SVD)라는 표준 행렬 분해 기법을 사용함.

$$\mathbf{V} = \begin{pmatrix} | & | & \cdots & | \\ \mathbf{c}_1 & \mathbf{c}_2 & & \mathbf{c}_n \\ | & | & | & | \end{pmatrix}$$

PCA는 데이터셋이 원점을 중심으로 중심화되어 있다고 가정함. Scikit-Learn의 PCA 클래스는 데이터 중심화를 자동으로 처리함. 직접 PCA를 구현하거나 다른 라이브러리를 사용하는 경우 먼저 데이터를 중심화해야 함.

d 차원으로 투영

모든 주성분을 식별한 후에는 첫 번째 d개의 주성분으로 정의된 초평면에 투영하여 데이터셋의 차원을 d 차원으로 줄일 수 있음. 이 초평면을 선택하면 투영이 가능한 한 많은 분산을 보존함.

- d 차원으로 축소된 데이터셋 X_{d-proj} 를 얻으려면 훈련 세트 행렬 X 와 행렬 W_d 의 행렬 곱을 계산함.
- W_d 는 V 의 첫 d개 열을 포함하는 행렬로 정의됨.

$$X_{d-proj} = XW_d$$

데이터셋의 차원을 가능한 한 많은 분산을 보존하면서 원하는 수의 차원으로 축소하는 방법임

Scikit-Learn 사용하기

Scikit-Learn의 PCA 클래스는 앞서 설명한 것처럼 SVD를 사용하여 PCA를 구현함. 다음 코드는 PCA를 적용하여 데이터셋의 차원을 2차원으로 축소함(데이터 중심화를 자동으로 처리함):

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X2D = pca.fit_transform(X)
```

PCA 변환기를 데이터셋에 맞춘 후, `components_` 속성은 W_d 의 전치를 포함함: 첫 번째 d 개의 주성분 각각에 대해 하나의 행을 포함함.



분산 보존

PCA는 데이터의 분산을 최대한 보존하는 방향으로 차원을 축소함



데이터 압축

차원 축소 후 훈련 세트는 훨씬 적은 공간을 차지하며 처리 속도가 빨라짐



역변환 가능

PCA로 축소된 데이터는 역변환을 통해 원래 차원으로 복원 가능함

설명된 분산 비율

각 주성분의 설명된 분산 비율은 `explained_variance_ratio_` 변수를 통해 확인할 수 있음.

```
array([0.7578477 , 0.15186921])
```

이 출력은 데이터셋 분산의 약 76%가 첫 번째 PC를 따라 있고, 약 15%가 두 번째 PC를 따라 있음을 알려줌. 세 번째 PC에는 약 9%가 남아 있으므로 세 번째 PC는 정보가 거의 없다고 가정할 수 있음.

적절한 차원 수 선택하기

임의로 차원 수를 선택하는 대신, 충분히 큰 분산 비율(예: 95%)을 합산하는 차원 수를 선택하는 것이 더 간단함.

MNIST 데이터셋의 경우 154개 차원이 분산의 95%를 보존하는 데 필요함

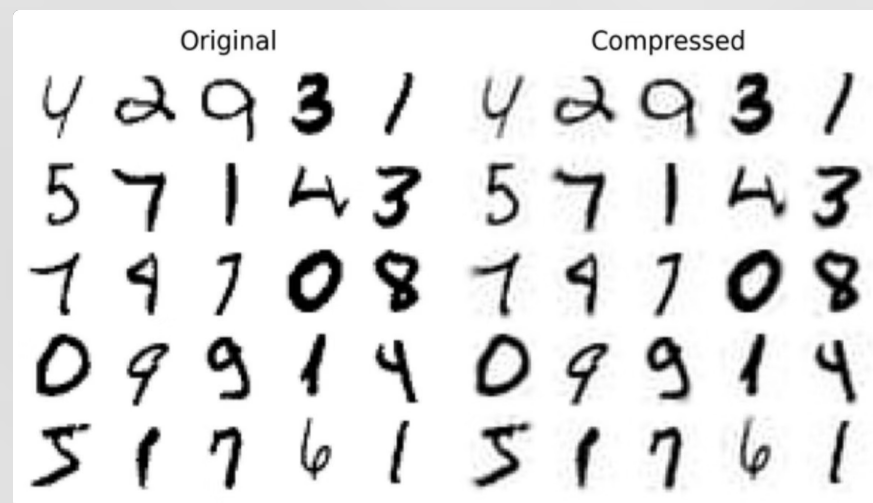
PCA를 이용한 압축

차원 축소 후 훈련 세트는 훨씬 적은 공간을 차지함. 예를 들어, MNIST 데이터셋에 PCA를 적용하여 분산의 95%를 보존한 후에는 원래 784개 특성 대신 154개 특성만 남음. 따라서 데이터셋은 원래 크기의 20% 미만이고 분산의 5%만 손실됨!

또한 PCA 투영의 역변환을 적용하여 축소된 데이터셋을 784차원으로 다시 압축 해제하는 것도 가능함. 이는 원본 데이터를 정확히 반환하지는 않지만(버려진 5% 분산 내에서 약간의 정보 손실) 원본 데이터와 유사함.

원본 훈련 세트의 몇 가지 숫자(왼쪽)와 압축 및 압축 해제 후의 해당 숫자를 보여줌. 약간의 이미지 품질 손실이 있지만 숫자는 대부분 그대로 유지됨.

$$X_{recovered} = X_{d-proj} W_d^T$$



랜덤 투영

이름에서 알 수 있듯이 랜덤 투영 알고리즘은 무작위 선형 투영을 사용하여 데이터를 저차원 공간으로 투영함. 이는 미친 소리처럼 들릴 수 있지만, 그러한 무작위 투영이 실제로 거리를 상당히 잘 보존할 가능성이 높음. 이는 William B. Johnson과 Joram Lindenstrauss가 유명한 보조정리에서 수학적으로 증명함.

차원을 많이 제거할수록 더 많은 정보가 손실되고 거리가 더 왜곡됨. 그렇다면 최적의 차원 수를 어떻게 선택할 수 있을까? Johnson과 Lindenstrauss는 거리가 주어진 허용 오차 이상으로 변경되지 않도록 보존해야 하는 최소 차원 수를 결정하는 방정식을 제시함.

$$d \geq \frac{4 \log(m)}{(\frac{1}{2}\epsilon^2 - \frac{1}{3}\epsilon^3)}$$

예를 들어, $m = 5,000$ 개의 인스턴스와 $n = 20,000$ 개의 특성을 가진 데이터셋이 있고, 두 인스턴스 간의 제곱 거리가 $\epsilon = 10\%$ 이상 변경되지 않도록 하려면 데이터를 $d = 7,300$ 차원으로 투영해야 함. 이는 상당한 차원 축소임!

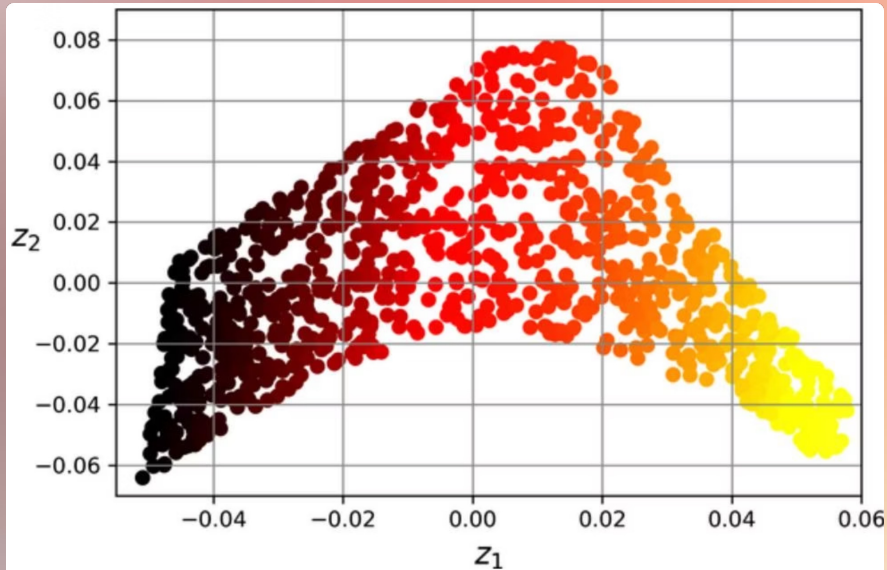
LLE (지역 선형 임베딩)

지역 선형 임베딩(LLE)은 비선형 차원 축소(NLDR) 기법임. PCA와 랜덤 투영과 달리 투영에 의존하지 않는 매니폴드 학습 기법임. 간단히 말해, LLE는 먼저 각 훈련 인스턴스가 가장 가까운 이웃과 선형적으로 어떻게 관련되는지 측정한 다음, 이러한 지역 관계가 가장 잘 보존되는 훈련 세트의 저차원 표현을 찾음.

이 접근법은 특히 노이즈가 너무 많지 않을 때 비틀린 매니폴드를 펼치는 데 특히 좋음. 다음 코드는 스위스 롤을 만든 다음 Scikit-Learn의 LocallyLinearEmbedding 클래스를 사용하여 이를 펼침:

```
from sklearn.datasets import make_swiss_roll
from sklearn.manifold import LocallyLinearEmbedding
X_swiss, t = make_swiss_roll(n_samples=1000,
                             noise=0.2, random_state=42)
lle = LocallyLinearEmbedding(n_components=2,
                             n_neighbors=10, random_state=42)
X_unrolled = lle.fit_transform(X_swiss)
```

결과 2D 데이터셋에서 볼 수 있듯이 스위스 롤이 완전히 펼쳐지고 인스턴스 간의 거리가 지역적으로 잘 보존됨. 그러나 더 큰 규모에서는 거리가 보존되지 않음. 그럼에도 불구하고 LLE는 매니폴드를 모델링하는 데 꽤 좋은 작업을 수행함.



기타 차원 축소 기법

다차원 척도법(MDS)

인스턴스 간의 거리를 보존하면서 차원을 축소함. 랜덤 투영은 고차원 데이터에서 이 작업을 수행하지만 저차원 데이터에서는 잘 작동하지 않음.

Isomap

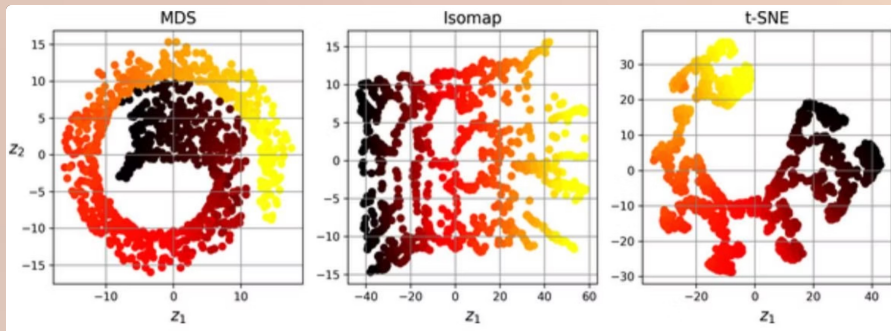
각 인스턴스를 가장 가까운 이웃에 연결하는 그래프를 만든 다음, 인스턴스 간의 측지 거리를 보존하면서 차원을 축소함. 그래프에서 두 노드 간의 측지 거리는 이 노드 사이의 최단 경로에 있는 노드 수임.

t-SNE

t-분포 확률적 이웃 임베딩은 유사한 인스턴스를 가깝게 유지하고 유사하지 않은 인스턴스를 멀리 떨어뜨리면서 차원을 축소함. 주로 시각화에 사용되며, 특히 고차원 공간에서 인스턴스 클러스터를 시각화하는 데 사용됨.

LDA

선형 판별 분석은 훈련 중에 클래스 간의 가장 판별적인 축을 학습하는 선형 분류 알고리즘임. 이러한 축은 데이터를 투영할 초평면을 정의하는 데 사용할 수 있음.



스위스 롤에 대한 MDS, Isomap 및 t-SNE의 결과: MDS는 전체적인 곡률을 잃지 않고 스위스 롤을 평평하게 만드는 반면, Isomap은 이를 완전히 제거함. t-SNE는 스위스 롤을 평평하게 만들고 약간의 곡률을 보존하며 클러스터를 증폭하여 롤을 분리함.

연습 문제

1. 데이터셋의 차원을 축소하는 주요 동기는 무엇인가? 주요 단점은 무엇인가?
2. 차원의 저주란 무엇인가?
3. 데이터셋의 차원이 축소되면 작업을 되돌릴 수 있는가? 가능하다면 어떻게? 불가능하다면 왜?
4. PCA를 사용하여 고도로 비선형적인 데이터셋의 차원을 축소할 수 있는가?
5. 1,000차원 데이터셋에 대해 설명된 분산 비율을 95%로 설정하여 PCA를 수행해 보자. 결과 데이터셋은 몇 차원을 가질 것인가?
6. 어떤 경우에 일반 PCA, 증분 PCA, 랜덤화된 PCA 또는 랜덤 투영을 사용하겠는가?
7. 데이터셋에서 차원 축소 알고리즘의 성능을 어떻게 평가할 수 있는가?
8. 두 개의 서로 다른 차원 축소 알고리즘을 연결하는 것이 의미가 있는가?

Thank You